

AWS Cloud Assignment — Multi-Team Report Processing Platform

Build a production-style cloud system using:

- AWS Lambda
- EventBridge
- RDS
- S3
- EC2
- ALB

The assignment is intentionally designed so students must create and use a **junction table** properly.

Problem Statement

Build a **Team Report Management System**.

Multiple employees can belong to multiple teams.

Each team uploads reports daily.

The system should:

- Upload reports to S3
 - Store metadata in RDS
 - Schedule automatic processing using Lambda + EventBridge
 - Host APIs on EC2 behind ALB
-

Real World Scenario

Example:

- Employee A works in:
 - Finance Team
 - Analytics Team
- Employee B works in:
 - Analytics Team
 - Operations Team

This creates a:

Many-to-Many Relationship

Between:

- Employees
- Teams

Students **MUST** solve this using a junction table.

Database Design Requirements

Students must create:

employees

```
CREATE TABLE employees (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(150) UNIQUE  
);
```

teams

```
CREATE TABLE teams (  
    id SERIAL PRIMARY KEY,  
    team_name VARCHAR(100)  
);
```

Junction Table (Mandatory)

employee_teams

```
CREATE TABLE employee_teams (  
    employee_id INT REFERENCES employees(id),  
    team_id INT REFERENCES teams(id),  
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (employee_id, team_id)  
);
```

Students MUST use:

- JOIN queries
- Composite primary keys
- Foreign keys

Reports Table

```
CREATE TABLE reports (  
    id SERIAL PRIMARY KEY,  
    team_id INT REFERENCES teams(id),  
    uploaded_by INT REFERENCES employees(id),  
    file_name VARCHAR(255),  
    s3_key VARCHAR(500),  
    status VARCHAR(50),  
    uploaded_at TIMESTAMP,  
    processed_at TIMESTAMP  
);
```

S3 Requirements

Create bucket:

```
team-report-storage
```

Folder structure:

```
/team-1/pending  
/team-1/processed  
  
/team-2/pending  
/team-2/processed
```

EC2 Backend API

Deploy backend on EC2.

Required APIs:

Employee APIs

```
POST /employees
GET /employees
```

Team APIs

```
POST /teams
GET /teams
```

Employee-Team Mapping APIs

These APIs MUST use the junction table.

```
POST /teams/:teamId/employees/:employeeId
GET /teams/:teamId/employees
GET /employees/:employeeId/teams
```

Students must write proper SQL JOIN queries.

Report APIs

```
POST /upload-report
GET /reports
GET /teams/:teamId/reports
```

Lambda Requirements

Create Lambda:

```
daily-team-report-processor
```

Lambda should:

1. Run every minute using EventBridge
 2. Fetch all pending reports
 3. Read files from S3
 4. Count rows/lines
 5. Update report status
 6. Move file:
 - pending → processed
-

EventBridge Requirement

Students must create cron-based scheduling.

Example:

```
cron(* * * * ? *)
```

ALB Requirements

Students must:

- Create Target Group
 - Attach EC2
 - Configure Listener
 - Access backend via ALB DNS
-

Mandatory SQL Queries

Students MUST demonstrate:

JOIN Query

Get all employees in a team:

```
SELECT e.name, t.team_name
FROM employees e
JOIN employee_teams et
ON e.id = et.employee_id
JOIN teams t
ON t.id = et.team_id;
```

Reverse JOIN Query

Get all teams for an employee.

Aggregation Query

Find:

- Number of reports uploaded per team
-

Mandatory AWS Concepts

Students must use:

- IAM Roles
 - Security Groups
 - CloudWatch Logs
 - Lambda Permissions
 - RDS Connectivity
 - ALB Routing
-

Bonus Tasks

Medium

- Add file upload validation
- Use presigned URLs

Hard

- Auto Scaling Group

- Multi-AZ RDS
- Retry failed reports
- Store processing logs in separate table

Deliverables

Students must submit:

1. ER Diagram
2. Architecture Diagram
3. GitHub Repository
4. SQL Schema
5. Screenshots:
 - Lambda
 - EventBridge
 - RDS Tables
 - ALB
 - S3 Bucket
6. API Documentation
7. Sample JOIN Queries

Evaluation Criteria

Topic	Marks
Proper Junction Table Usage	20
SQL JOIN Queries	15
Lambda + EventBridge	15
S3 Integration	10
EC2 + ALB	15
RDS Design	15
IAM + Security	10

Expected Learning Outcome

Students will learn:

- Many-to-many DB design
- Junction tables
- SQL JOINS
- AWS serverless workflows
- Backend deployment
- Load balancing
- Scheduled automation
- Production-grade architecture